

Federated QoS Offload Decisions for OneM2M IoT Middleware

Project Report

MSc Research Project / (Research) Practicum / Internship
Cloud Computing

Uday
Student ID: TBD

School of Computing
National College of Ireland

Supervisor: TBD

September 2026

Federated QoS Offload Decisions for OneM2M IoT Middleware

Uday
TBD

Abstract

Et-Tousy, Zyane & Sharif (2026), “Adaptive QoS Management in OneM2M Standard: Machine Learning and Deep Learning for IoT Network Optimization” (Journal of Network and Systems Management), trains a centralized Random Forest on pooled QoS telemetry (RTT, CPU, RAM, success rate) to decide whether IoT traffic should stay local, partially offload, or fully offload to the cloud, reaching 98% accuracy. The paper explicitly names a gap it does not resolve: its centralized learning mechanism “might not scale well or guarantee data privacy in distributed IoT environments,” with federated learning proposed only as future work. This project reproduces the centralized baseline on synthetic QoS telemetry from 6 non-IID simulated IoT edge sites (each dominated by one of the paper’s own uniform/burst/real-time traffic scenarios), then builds and evaluates the federated alternative the paper names but does not build: one Random Forest per site, trained on local data only, combined at inference time by averaging predicted probabilities, so no raw telemetry ever leaves its site. Across 5 seeds, the federated ensemble reaches 99.5% accuracy and 99.5% macro-F1 versus the centralized baseline’s 99.9%/99.96% – a small utility cost for removing the need to centralize raw data – while a single site’s own local model used in isolation, with no federation at all, reaches only 97.6% accuracy and 97.4% macro-F1. Federation recovers almost all of the gap between going it alone and pooling everything centrally, without requiring the centralization the baseline paper’s own limitations section flags as a privacy risk.

1 Introduction

IoT middleware platforms built on standards like OneM2M need to decide, in real time, whether to process a device’s traffic locally, offload part of it to the cloud, or fully migrate processing to the cloud, based on current network and resource conditions. Static, manually-configured rules cannot keep up with dynamic, bursty IoT traffic, motivating machine-learning-driven QoS management.

Et-Tousy, Zyane & Sharif Et-Tousy et al. (2026) address exactly this with a centralized Random Forest classifier trained on pooled QoS telemetry (round-trip time, CPU, RAM, success rate) from all connected devices, achieving 98% accuracy at recommending Keep-Local / Partial-Offload / Full-Offload decisions. Their own Discussion section names a limitation they do not resolve: centralizing telemetry from every device “might not scale well or guarantee data privacy in distributed IoT environments,” proposing federated learning only as future work.

Research question: can a federated learning alternative – training one model per IoT site on local data only, and combining them without ever pooling raw telemetry – recover most of the accuracy of the centralized baseline, while removing the need to centralize data at all?

Objectives:

- Reproduce the baseline paper’s centralized Random Forest classifier and its exact decision thresholds and class-imbalance handling (SMOTE), on synthetic telemetry from simulated, non-IID IoT sites.
- Build the federated alternative the paper names as future work but does not build: per-site local Random Forests combined at inference time.
- Quantify what a site loses by going it alone (no federation, no centralization) as a lower-bound comparison, to establish whether federation is actually worth building relative to that baseline.
- Evaluate all three variants on the same held-out, pooled test set across multiple random seeds.

The remainder of this report is structured as follows: Section 2 reviews the baseline paper and the specific gap it names; Section 4 describes the reproduced baseline and the federated design; Section 5 describes the synthetic multi-site telemetry generator, training pipeline, and deployment; Section 6 reports results across 5 seeds; Section 7 discusses limitations and future work.

2 Related Work

IoT middleware standards such as OneM2M provide interoperability across vendors and device types, but leave dynamic Quality-of-Service (QoS) management and traffic-offload decisions largely unaddressed, forcing operators to rely on manual, reactive intervention when systems become overloaded.

2.1 Baseline: ML/DL for OneM2M QoS Optimization

The baseline for this project is Et-Tousy, Zyane & Sharif Et-Tousy et al. (2026), “Adaptive QoS Management in OneM2M Standard: Machine Learning and Deep Learning for IoT Network Optimization” (Journal of Network and Systems Management, May 2026). The paper collects real-time QoS telemetry (RTT, CPU, RAM, success rate) from Azure IoT devices via a MAPE-K monitoring loop, labels each window as Preferable / Acceptable / Critical using fixed thresholds, and trains classical ML models (Random Forest, SVC, KNN, XGBoost, LightGBM), ensembles (Voting/Stacking of RF+LightGBM), and deep learning models (CNN, LSTM, VAE+RF) to recommend a traffic-offload decision: keep processing local, partially offload to the cloud, or fully offload. Their best model (a Random Forest / LightGBM ensemble) reaches 98% accuracy and is deployed as a live API that drives the Analyzer stage of a MAPE-K loop, reducing RTT by 30-50% and increasing success rates by 10-15% across uniform, burst, and real-time traffic scenarios.

2.2 The Gap This Project Targets

The paper is explicit about what it does not address. In their own Discussion section: “the system currently relies on a centralized learning mechanism, which might not scale well or guarantee data privacy in distributed IoT environments. To overcome this, we plan to explore federated learning.” This is stated as future work, not built or evaluated in the paper itself.

2.3 Federated Learning for Edge/IoT Systems

Federated learning is a well-established response to exactly this concern – training models across distributed clients without centralizing raw data – but the baseline paper’s own centralized Random Forest pipeline does not use it. This project reproduces that centralized baseline, then builds and evaluates a federated alternative: one Random Forest per simulated IoT site, trained only on that site’s local telemetry, combined at inference time by averaging predicted class probabilities across sites. No raw telemetry is pooled across sites at any point, directly addressing the privacy and centralization concern the baseline paper names but does not resolve.

3 Methodology

3.1 Synthetic Multi-Site QoS Telemetry

Real multi-site OneM2M telemetry was not available, so a synthetic generator (`src/data/qos_simulator`) produces QoS records (RTT, CPU, RAM, success rate, latency, throughput) for 6 simulated IoT edge sites. Each site’s traffic is a mixture of the baseline paper’s own three scenarios – uniform, burst, and real-time – but with a different, skewed mixture per site (80% dominated by one scenario, 10% each of the other two), which is what makes this a genuine non-IID federated learning setting: no single site sees a representative sample of the overall traffic distribution. 1,500 records per site are generated per random seed.

3.2 Training

For each of 5 seeds (42-46), each site’s data is split 80/20 (stratified by label) into a local train/test set. The centralized model pools all sites’ training splits and SMOTE-balances the pooled result before fitting. The federated model fits one Random Forest per site on that site’s training split only, SMOTE-balanced locally. Both use identical Random Forest hyperparameters (200 trees, max depth 10) so any performance difference reflects the data-sharing strategy, not model capacity. All three variants (centralized, federated, single-site-only) are evaluated on the same pooled, held-out test set (20% of every site’s data, never used in training by any model).

3.3 Metrics

- **Accuracy / Macro-F1:** over the full 3-class pooled test set.
- **Critical Recall (class 2):** recall specifically on the “Full Offload” class – the minority class the baseline paper needed SMOTE to handle (9.64% of their real

data), and the one most consequential to miss in production (a missed Critical event means an overloaded system stays local).

All code and results are in `iot-reliability/`.

4 Design Specification

4.1 Decision Labels and Thresholds

Each QoS record is labelled using the baseline paper’s own thresholds: *Keep Local* (Preferable) when $RTT < 2s$ with healthy resource usage and success rate; *Partial Offload* (Acceptable) when RTT is between 2s and 4s; *Full Offload* (Critical) when RTT exceeds 4s, success rate drops below 90%, or CPU/RAM usage exceeds 80%.

4.2 Centralized RF (baseline reproduction)

All sites’ raw QoS telemetry (RTT , CPU, RAM, success rate, latency, throughput) is pooled into one training set, class-balanced with SMOTE (matching the paper’s methodology for its reported 48%/42%/10% class split), and a single Random Forest (200 trees, max depth 10) is trained on the pooled, balanced data – reproducing the model the paper actually selected for production deployment.

4.3 Federated Ensemble RF (proposed improvement)

Each of the 6 simulated IoT sites trains its own Random Forest using the same hyperparameters, on its own local telemetry only, SMOTE-balanced locally. At inference time, the site models’ predicted class probabilities are averaged and the highest-probability class is returned. No site’s raw telemetry is ever transmitted to or pooled with another site’s data, or with a central server – only the independently-trained models are combined.

4.4 Single-Site Local Only (lower bound)

To establish whether federating is actually worth the engineering effort, each site’s own local model is also evaluated in isolation (no combination with any other site) against the same held-out global test set. This represents what an individual IoT gateway is stuck with if it can neither share data nor collaborate on a model with any other site.

5 Implementation

The system is implemented in Python 3 with scikit-learn and imbalanced-learn. Key modules:

- `src/data/qos_simulator.py` – synthetic multi-site QoS telemetry generator described in Section 3.
- `src/models/qos_models.py` – `CentralizedRF` (baseline) and `FederatedEnsembleRF` (improvement), both wrapping scikit-learn’s `RandomForestClassifier` with SMOTE balancing via imbalanced-learn.

- `scripts/train_and_evaluate.py` – trains and evaluates all 3 variants across 5 seeds, writes `results/results_per_seed.csv` and `results/results_summary.csv`, and exports one trained model per site (via joblib) to `src/lambda_handler/model/` for deployment.
- `src/lambda_handler/app.py` – an AWS Lambda handler that loads all 6 exported per-site models and applies the federated ensemble decision to each incoming QoS telemetry record arriving on a Kinesis stream, returning a Keep-Local/Partial-Offload/Full-Offload decision with class probabilities.
- `template.yaml` – AWS SAM template provisioning the Kinesis stream and Lambda function.
- `.github/workflows/deploy.yml` (repo root) – CI pipeline that runs the training/evaluation script and, on push to `main`, deploys the SAM stack to the configured AWS region using repository secrets.

Running `python scripts/train_and_evaluate.py` reproduces every number in Section 6 from scratch. The Lambda handler was verified locally against representative telemetry for all three decision classes before deployment.

6 Evaluation

Table 1 reports mean $\pm$ standard deviation across 5 seeds (42-46) on a pooled, held-out test set drawn from all 6 sites.

Table 1: Results across 5 seeds (mean $\pm$ std)

Model	Accuracy	Macro-F1	Critical Recall (class 2)
Centralized RF (baseline)	0.9995 ± 0.0005	0.9996 ± 0.0005	0.9994 ± 0.0013
Federated Ensemble RF (improved)	0.9959 ± 0.0024	0.9949 ± 0.0024	0.9997 ± 0.0007
Single-Site Local Only (no federation)	0.9761 ± 0.0046	0.9737 ± 0.0051	0.9958 ± 0.0031

6.1 Experiment 1: The Cost of Going It Alone

A single site’s own local model, used in isolation with no federation and no centralization, reaches only 97.4% macro-F1 – a clear, consistent gap below both the centralized (99.96%) and federated (99.49%) variants. This confirms the premise of the whole exercise: because each site’s local traffic mixture is skewed toward one of the three scenarios (uniform/burst/real-time), a site that only ever sees its own data generalises worse to the full population it will actually encounter in production.

6.2 Experiment 2: Federation Recovers Most of the Centralization Benefit

The federated ensemble closes the large majority of the gap between local-only and centralized (macro-F1 $97.37\% \rightarrow 99.49\%$, versus centralized’s 99.96%), while never pooling raw telemetry across sites – directly addressing the privacy/centralization limitation the

baseline paper names in its own Discussion section. The remaining gap to full centralization (99.49% vs. 99.96% macro-F1, about half a point) is the measurable cost of keeping data decentralized, and is small relative to the 2.6-point gap federation recovers from the single-site baseline.

6.3 Experiment 3: The Minority Class

Critical Recall (the class the baseline paper itself needed SMOTE to handle, at only 9.64% of their real-world data) is high and statistically indistinguishable across all three variants (99.6-100%), including the single-site baseline. This suggests SMOTE-balancing within each site is sufficient on its own to protect the minority class here; federation’s main benefit in this experiment is on overall accuracy/macro-F1 (driven by better generalisation across traffic regimes), not on minority-class recall specifically.

6.4 Discussion

All three variants score unusually high in absolute terms (97-100%) compared to the baseline paper’s own reported 96-98% for its best models. This is expected: the synthetic generator’s labelling rule is a deterministic function of the same features the models are trained on, with less real-world sensor noise than Azure IoT telemetry would have. The relative ordering of the three variants – local-only worst, federated in the middle, centralized best – is the finding that matters here, not the absolute numbers, and that ordering is what a privacy/utility trade-off in federated learning would predict.

7 Conclusion and Future Work

This project asked whether a federated learning alternative to Et-Tousy et al.’s (2026) centralized Random Forest – one model per IoT site trained on local data only, combined without pooling raw telemetry – could recover most of the centralized model’s accuracy while resolving the privacy/scalability limitation the paper names but does not build. Across 5 seeds on synthetic, non-IID multi-site QoS telemetry, the federated ensemble reached 99.49% macro-F1 versus the centralized baseline’s 99.96% and a single site’s own local-only model’s 97.37% – recovering the large majority of the benefit of centralization without requiring it.

The clearest finding is not the small centralized-vs-federated gap, but the large local-only-vs-federated gap: a site that cannot collaborate with any other site (no data sharing, no federation) generalises measurably worse to the traffic patterns it will actually see in production. Federation is the difference between a site being stuck with a locally-biased model and getting most of the benefit of the full population’s data, without that data ever leaving its site of origin.

Future work:

- Evaluate on real multi-site OneM2M/Azure IoT telemetry rather than synthetic data, to check whether the federation benefit holds under real sensor noise and real non-IID splits.
- Extend the federated design from simple probability-averaging to a weighted aggregation scheme (e.g. weighting each site’s vote by its local validation confidence), which the current implementation does not attempt.

- Evaluate secure aggregation (so that not even model updates are individually inspectable by a central coordinator), extending the privacy property this project targets but does not fully secure.

References

Et-Tousy, J., Zyane, A. and Sharif, S. (2026). Adaptive QoS management in OneM2M standard: Machine learning and deep learning for IoT network optimization, *Journal of Network and Systems Management* . Preprint: <https://doi.org/10.21203/rs.3.rs-7987618/v1>.